



PROJET DE BASES DE DONNÉES

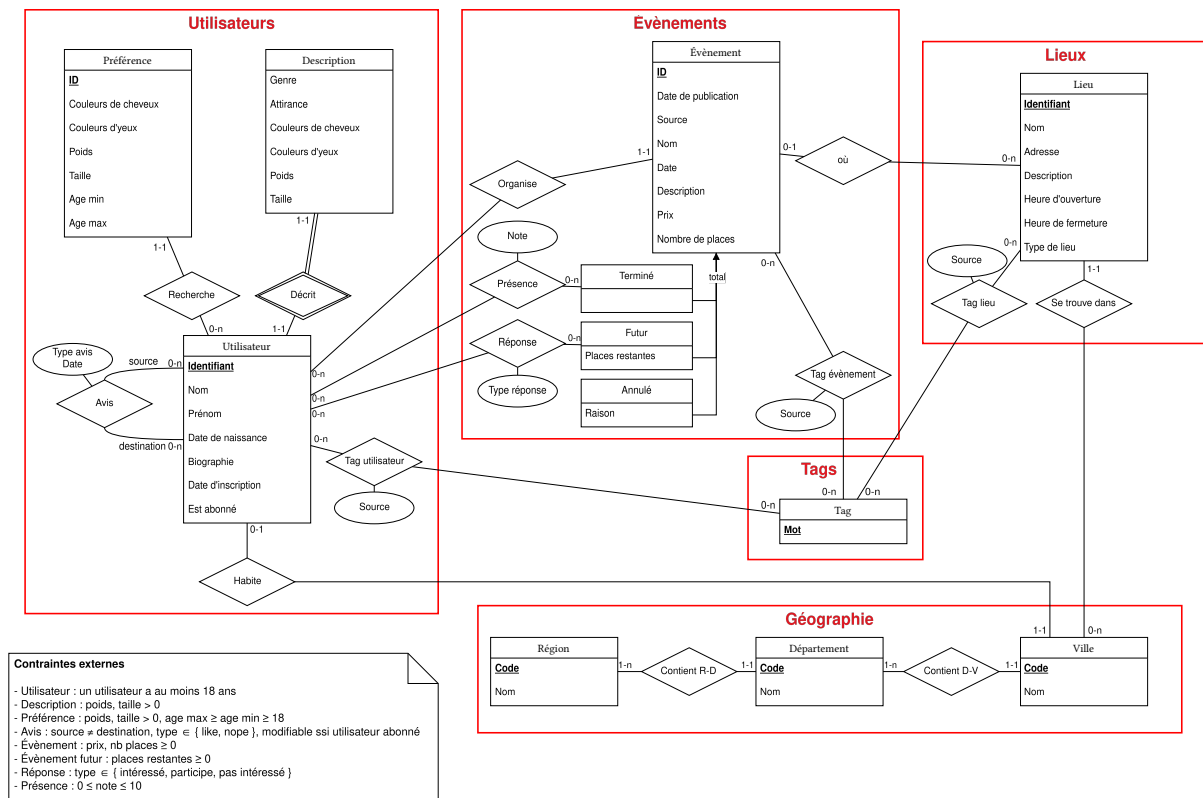
Le Big Match

Anthony Fernandes

Marc Robin

I. Modélisation ER

Voici le diagramme ER que nous avons établi à partir du sujet et en fonction duquel la base de donnée a été construite :



Note : Si l'image vectorielle ne s'affiche pas correctement, il y a aussi le fichier [diagramme_ER.png](#)

Nous avons découpé les informations du sujet 4 catégories : ce qui concerne les utilisateurs, les événements, les lieux et les tags. Nous avons également ajouté une 5ème catégorie concernant la géographie pour pouvoir évaluer la distance entre deux villes.

La catégorie la plus simple est justement cette catégorie de géographie. Elle contient une représentation simplifiée de la France avec ses villes, départements et régions imbriqués. Cette modélisation permet deux choses : on peut grossièrement estimer la proximité entre deux villes (même si ça fonctionne en réalité mal pour les villes en bordure de département/région) ; et on peut proposer aux utilisateurs de l'appli une auto complétion des champs de villes.

Concernant la catégorie des utilisateurs, la plus importante, elle contient évidemment l'entité centrale `Utilisateur` qui représente un utilisateur de l'application avec ses informations personnelles et les variables internes de l'appli comme la date d'inscription et le status de l'abonnement. Cette entité est également liée à la ville où déclare habiter l'utilisateur. Pour ne pas trop charger cette entité (et la table SQL qui en résulte), la description physique des utilisateurs est stockée dans l'entité faible `Description`, elle est faible car rien à part son leur utilisateur associé ne permet de d'identifier de façon unique les descriptions. Un utilisateur peut ensuite exprimer plusieurs préférences/critères de recherche concernant les profils qu'il souhaite rencontrer. Ces préférences sont représentée par l'entité du même nom.

Le système de like/nope est représenté par l'association réflexive `Avis` qui s'interprète comme « `<source>` a liké/nopé `<destination>` », le type de l'avis indique s'il s'agit d'un like ou d'un nope. Une condition externe implémentable en SQL avec un check a été précisée sur cette relation pour empêcher d'un utilisateur de se donner un avis à soit-même.

Ces utilisateurs assistent à des événements ayant diverses caractéristiques : nom, date, description, nombre de place, organisateur... Comme les événements peuvent également être importés par l'application depuis des sites externes, l'attribut `source` permet de garder une trace de la provenance d'un événement. On utilise `1bm` pour les événements déclarés depuis l'application. On a distingué 3 types d'événements formant un héritage total et disjoint : les événements à venir, qui ont un nombre de places restantes et auxquels les utilisateurs peuvent informer de leur présence (`assiste`, `interesse` ou `pas_interesse`) ; les événements terminées auxquels les utilisateurs peuvent laisser une note de 0 à 10 ; enfin les événements annulés ayant une raison de l'annulation.

Les événements se passent eux mêmes dans des lieux ayant un no, une adresse dans une ville, une description, des horaires...

Enfin la dernière catégorie concerne les tags. Ils s'agit simplement d'une seule entité avec un seul attribut pour le mot clé associé à ce tag. Les trois entités principales (utilisateur, événement, lieu) peuvent être associées à un nombre arbitraire de tags. On se sert également du système de tags pour stocker les informations inférées concernant un utilisateur/événement/lieu. Par exemple, si un utilisateur like des musiques de jazz sur spotify, on lui inferera le tag jazz ainsi que le titre des morceaux et du nom des artistes. L'attribut `source` permet, comme pour les événements, de garder une trace de la provenance d'un tag.

II. Schéma relationnel

Le schéma relationnel est essentiellement une traduction assez naturelle du diagramme ER. Pour l'élimination de l'héritage des événements, nous avons opté pour conserver toutes les entités et faire des entités filles des entités faibles. Nous avons fait ce choix car en éliminant l'entité `Evenement`, tout ses attributs auraient été reportés dans les filles et cela aurait pu induire plus d'erreurs recopie quand les événements qui viennent de passer sont déplacés chaque jour dans la table appropriée ; au contraire, éliminer les entités filles n'aurait pas permis de garantir que les tables `presence` et `reponse` se réfèrent bien à des événements passés et futurs. Pour aider à maintenir la cohérence entre les différentes tables d'événements (un événement est exactement dans une table fille à la fois, les événements de la table futur sont réellement futurs...), nous avons fourni une requête qui renvoie exactement les lignes incohérentes.

Concernant les tags, nous n'avons pas créé de table pour cette entité. En effet, elle ne se compose que d'une unique clé qui est reportée dans toutes les associations auxquelles l'entité `Tag` participe, il n'aurait donc pas de sens de garder cette table inutile.

On a finalement agrémenté le schéma de deux vues qui semblent pertinentes dans le cadre de cette application : la vue `util_desc` qui fait un left join entre `utilisateur` et `description` pour pouvoir simultanément accéder aux données personnelles d'un utilisateur et à sa description physique, et la vue `match` qui extrait de la table `avis` les couples d'utilisateurs qui se sont likés mutuellement.

III. Algorithme de matching

Voyons maintenant comment nous avons choisi d'exploiter notre schéma dans le but de faire matcher les utilisateurs entre eux. Il est clair qu'il n'y a pas une unique bonne manière de faire et que nous avons choisi une méthode parmi tant d'autres.

L'idée qu'on a finalement retenue, qui est d'ailleurs l'une des premières qui nous est venue à l'esprit, repose sur un principe assez intuitif : étant donné un utilisateur, on attribue une note de compatibilité, comprise entre 0 et 1, à chaque autre personne qui pourrait potentiellement lui correspondre. Cette note permet ensuite d'estimer dans quelle mesure deux profils pourraient s'entendre, en se basant sur différents critères présents dans notre base.

Cette approche a l'avantage de permettre une comparaison directe entre plusieurs profils, et donc de classer facilement les utilisateurs selon leur « degré de compatibilité » avec une personne donnée.

Comment détermine-t-on les notes attribuées aux matchs potentiels ?

Le Big Match étant un site de rencontre très axé sur les événements il nous a semblé logique de leur accorder un poids important dans le calcul de la compatibilité.

Les tags, ont eux aussi leur rôle à jouer, même si nous leur avons donné une importance un peu plus modérée. Ils permettent de refléter les centres d'intérêt ou les traits de personnalité, et contribuent à affiner la compatibilité entre deux utilisateurs. C'est à ce titre que nous avons choisi de donner 50% de la note à cet aspect de notre réseau réparti de la manière suivante : 30% pour les événements et 20% pour les tags.

Les scores de tags sont calculés de la manière suivante : Si T_1 (resp. T_2) est l'ensemble des tags de u_1 (resp. u_2) alors le score de tags est donné par $\frac{|T_1 \cap T_2|}{|T_1 \cup T_2|}$, qui représente le nombre de tags qu'ils ont en commun divisé par le nombre de tags qu'ils ont fait en cumulé. Plus ils ont de tags en commun plus ils ont de chance d'être compatibles. Nous avons procédé de la même manière pour déterminer le score associé aux événements.

Maintenant que nous avons pris en compte les particularités de notre site de rencontre, intéressons-nous aux préférences de chaque utilisateur car même si deux personnes ont plein de points communs, si l'une d'elles ne correspond pas du tout aux attentes de l'autre, ça ne pourra jamais être un bon match. Les affinités, c'est important, mais les préférences individuelles le sont tout autant c'est donc pour cette raison que nous avons réservés les 50% restants aux préférences des utilisateurs.

Une première sélection est faite en fonction des attirances. Il va de soi que si un utilisateur est attiré par les garçons, on ne va pas lui proposer une fille, ni même un garçon qui, de son côté, n'est pas attiré par le sexe de cet utilisateur. Pour des raisons pratiques nous n'avons considérés que homosexuel, bisexuel et hétérosexuel. De même si deux personnes sont trop éloignées alors elles ne sont pas considérées comme compatibles.

Ensuite nous déterminons une note comprise entre 0 et 1 pour chaque personne étant compatible (au sens des critères du paragraphe ci-dessus).

Chaque utilisateur a la possibilité de renseigner ses préférences, la taille, la couleur d'yeux et de cheveux, le poids un âge minimum et maximum.

Nous avons considéré que l'âge est le critère le plus important et qu'il est primordial que ce critère soit central au sein même des préférences. On le lui accorde donc 50% et repartissons de manière équitable le reste c'est à dire 12.5% chacun.

Nous sommes conscient que cette pondération n'est probablement pas idéal mais à le bon goût d'être assez malléable. Ainsi il est dans nos plans de proposer aux utilisateurs abonnés de pouvoir choisir eux même ce qui leur semble important et ce qu'ils veulent favoriser.

Un utilisateur n'est pas obligé de renseigner de fiche de préférence. Dans ce cas nous avons choisi de mettre le score de préférence à 1. Ce choix ne change en rien le classement final car tout le monde est le soumis à la même convention et donc les départages sont fait sur les tags et évènements. Le score associé à la couleur des yeux et des cheveux vaut 1 si c'est exactement la bonne couleur.

C'est pour les autres critères que nous avons une méthode un peu plus recherchée. Il n'est pas raisonnable pour des raisons évidentes de faire pareil que précédemment pour la taille, le poids et l'âge. Nous voulions que plus l'élément considéré est proche de celui voulu plus le score est élevé et que plus nous nous en éloignons plus le score baisse. Nous voulions également que l'écart compté positivement ou négativement ai le même impact sur le score. La dernière contrainte que nous voulions était une décroissante assez lente au début puis assez rapide et que passé un seuil elle vale zéro.

Traduisons cela en terme un peu plus mathématique. Nous cherchons une fonction $f : \mathbb{R} \rightarrow \mathbb{R}$, paire, décroissante sur $\mathbb{R}_+$, telle que $f(0) = 1$, si $x \in \mathbb{R}_+$, $x \geq a$ alors $f(x) = 0$ où a représente le seuil fixé selon le critère.

Ainsi de ces contraintes nous avons extrait la fonction suivante :

$$f : \mathbb{R} \rightarrow \mathbb{R},$$
$$x \mapsto \begin{cases} 1 - \left(\frac{x}{a}\right)^2 & \text{si } |x| \leq a \\ 0 & \text{sinon} \end{cases}$$

Le coefficient a est fixé selon le critère considéré. Il est fixé à 10 pour la taille et le poids. Quant à l'âge nous l'avons fixé à 5 avant de finalement choisir de mettre de borne inf et sup et de mettre le score de l'âge à 1 si et seulement s'il est exactement entre les deux bornes car cela nous semblait plus judicieux et plus précis. Nous avons gardé ce système pour la taille et le poids car ce sont des critères moins sensibles aux variations.

IV. Limitations

Nous avons réservé une catégorie entière aux tags car il est possible de les étendre d'avantage avec des tags pour référencer un utilisateur, un lieu ou un évènement. Regardez par exemple la première version de notre diagramme ER (*vous pouvez aussi consulter le fichier [ancien_diagramme.png](#)*) :



Ce schéma ressemble au schéma actuel à deux exceptions notables. Tout d'abord, nous avons implémenté une hiérarchie complète de tags mais nous nous sommes rendu compte qu'à l'étape de traduction en SQL, une telle architecture engendrerait beaucoup trop de tables. La seconde différence est la présence de l'entité `Intérêt` dans la catégorie utilisateur. Nous avons initialement décidé de représenter les intérêts des utilisateurs comme leur propre entité avec un système d'héritage pour chaque centre d'intérêt pris en compte par l'application. Un problème c'est alors posé dans la planification de l'algorithme de matching : si l'héritage est éliminé par élimination de l'entité mère alors il y aurait une table par centre d'intérêt à prendre en compte dans les calculs, rendant l'appli très peu extensible, en revanche, nous n'avons pas réussi non plus à trouver un moyen suffisamment générique pour regrouper tous les centres d'intérêts dans une seule entité.